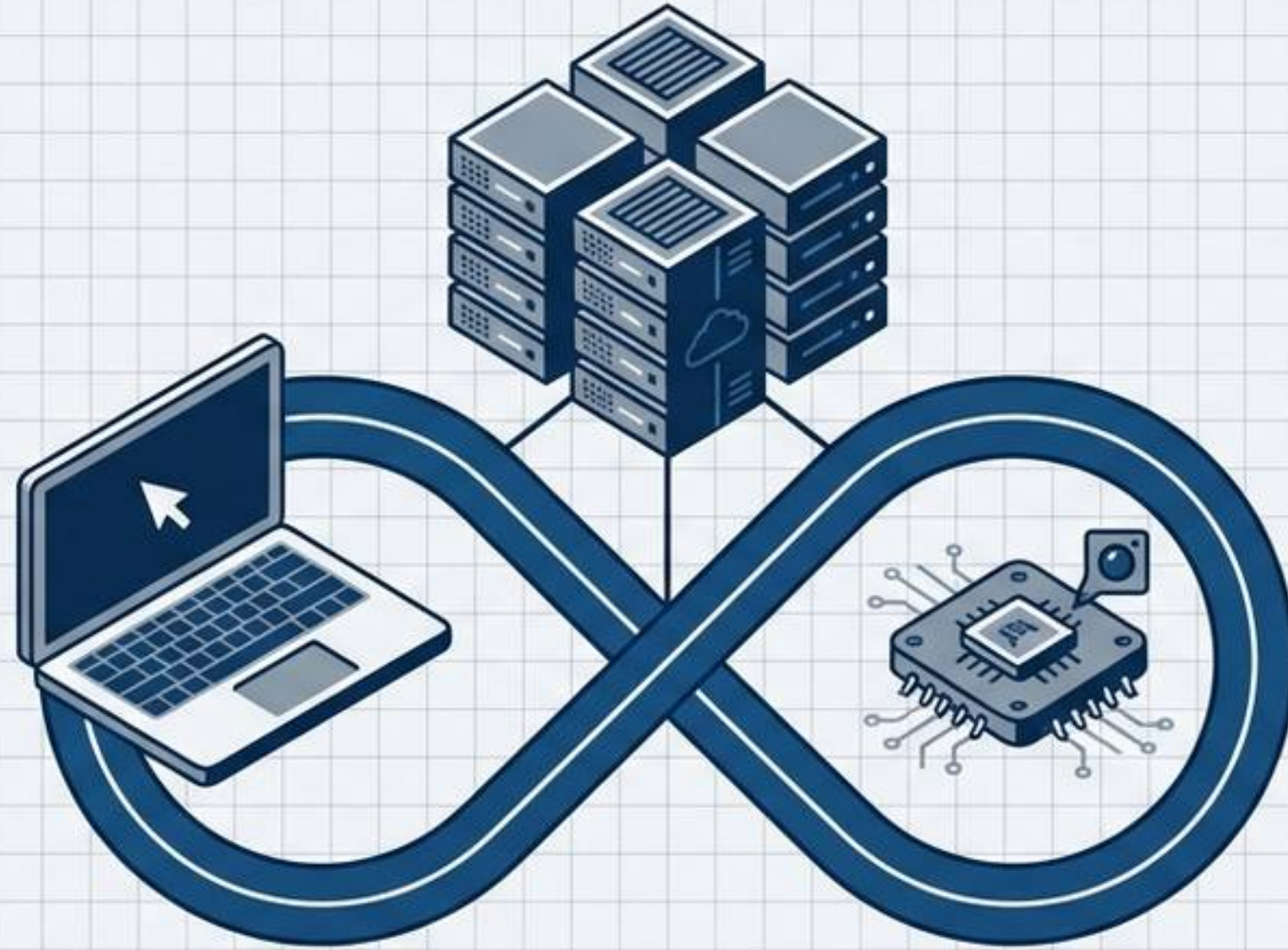
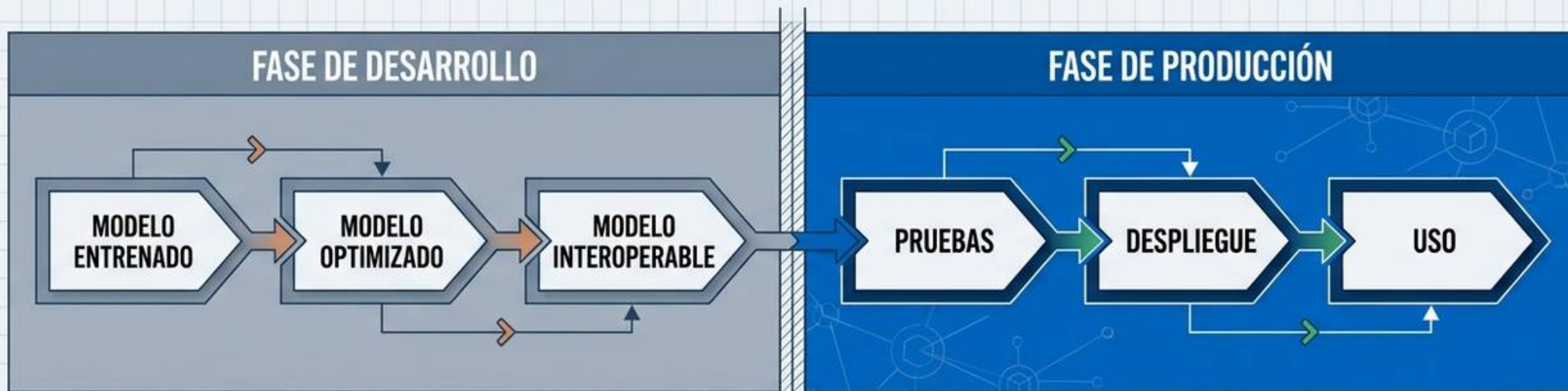




DESPLIEGUE DE MODELOS E INTEGRACIÓN CONTINUA

SEMANA 17 | MACHINE LEARNING 2026: DEL LABORATORIO AL ENTORNO DE PRODUCCIÓN

DE LA EXPERIMENTACIÓN AL MUNDO REAL



EL CAMBIO DE PARADIGMA

En producción no entrenamos modelos; ejecutamos inferencia con un modelo ya aprendido.

¿QUÉ ES EL DESPLIEGUE?

El traslado operativo de una solución desde el entorno de desarrollo cerrado hacia el usuario final.

EL PAQUETE COMPLETO

Desplegar no es solo exportar el archivo del modelo. Implica empaquetar de forma conjunta: **La Aplicación + El Modelo + Las Dependencias + El Entorno.**

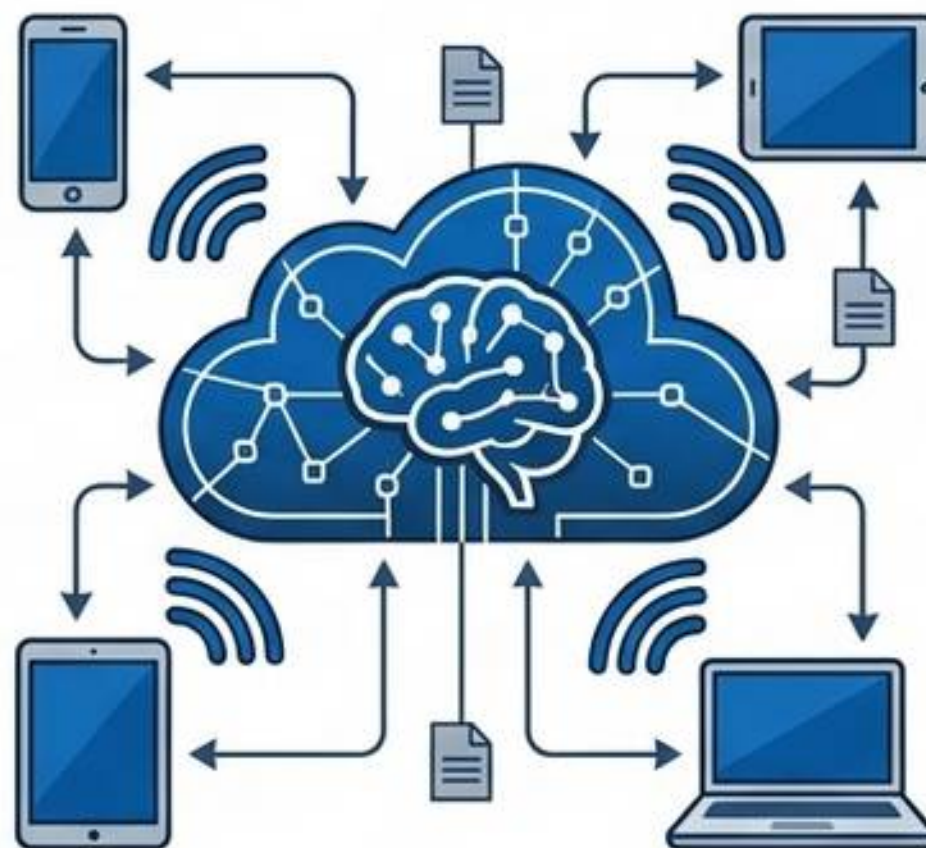
¿DÓNDE VIVIRÁ NUESTRO MODELO?

DESPLIEGUE LOCAL



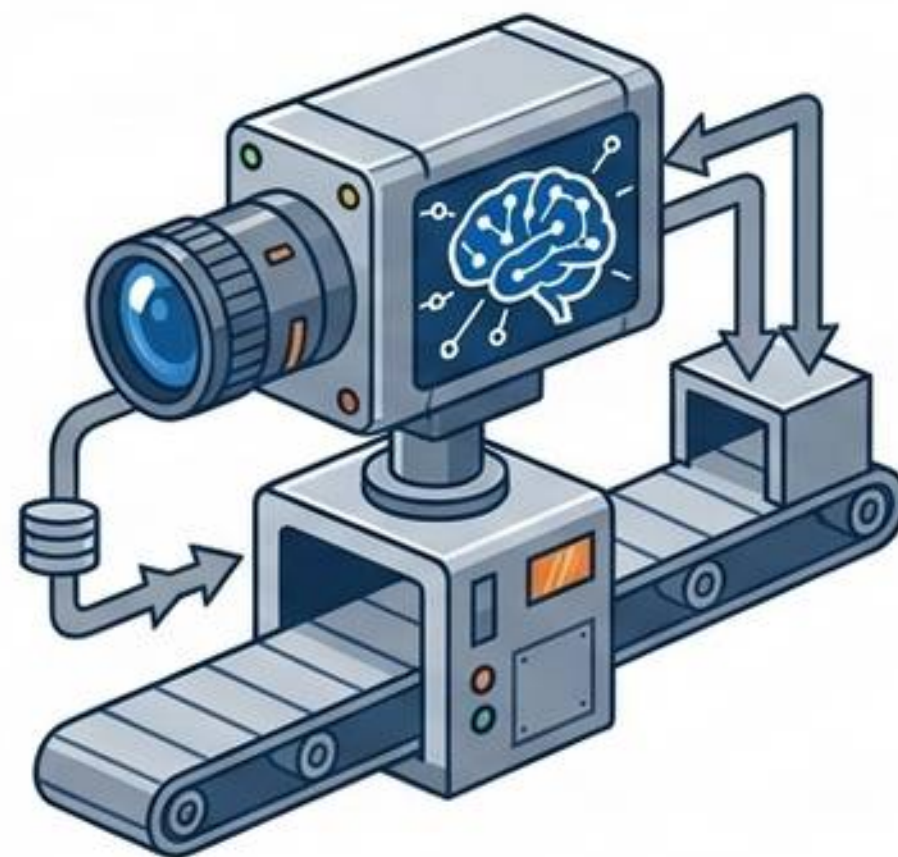
Aplicación y modelo operan en la infraestructura del propio usuario (ej. su computadora personal).

DESPLIEGUE CLOUD (NUBE)



El modelo reside en servidores remotos y altamente escalables, accesibles únicamente vía red.

EDGE COMPUTING (BORDE)



El modelo opera directamente en dispositivos con recursos limitados físicamente cercanos a la fuente de datos.

DESPLIEGUE LOCAL: SIMPLICIDAD Y PRIVACIDAD



Concepto Principal

Todos los componentes residen físicamente en el equipo del usuario final (ej. aplicación de Streamlit offline).

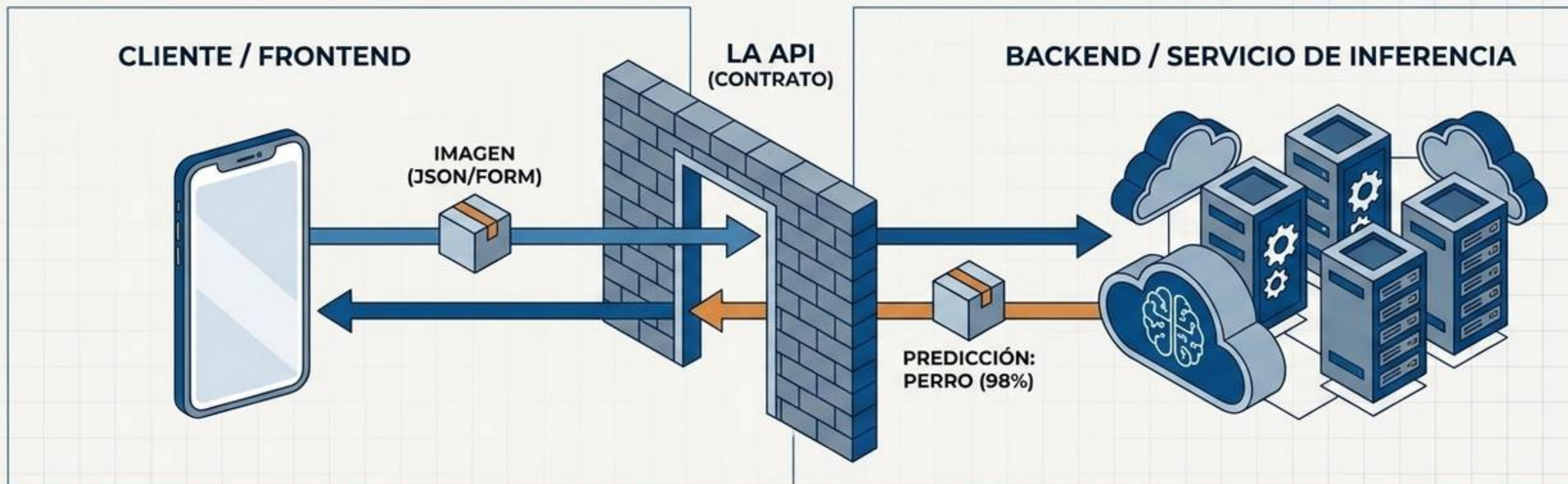
✓ Ventajas Estratégicas

- **Privacidad absoluta:** Los datos (imágenes) nunca salen del dispositivo.
- **Operación Offline:** Nula dependencia de conexión a Internet.
- **Cero costos cloud:** No se paga por infraestructura o servidores remotos.

⚠ Desafíos Operativos

- **Distribución compleja:** Requiere instalar Python, bibliotecas y el modelo en cada máquina cliente.
- **Restricciones de Hardware:** El rendimiento de la inferencia depende totalmente de la CPU/GPU local del usuario.

DESPLIEGUE CLOUD: CENTRALIZACIÓN A TRAVÉS DE APIS



ARQUITECTURA DESACOPLADA

El dispositivo del cliente es ligero; no necesita almacenar ni tener capacidad para ejecutar la red neuronal pesada.

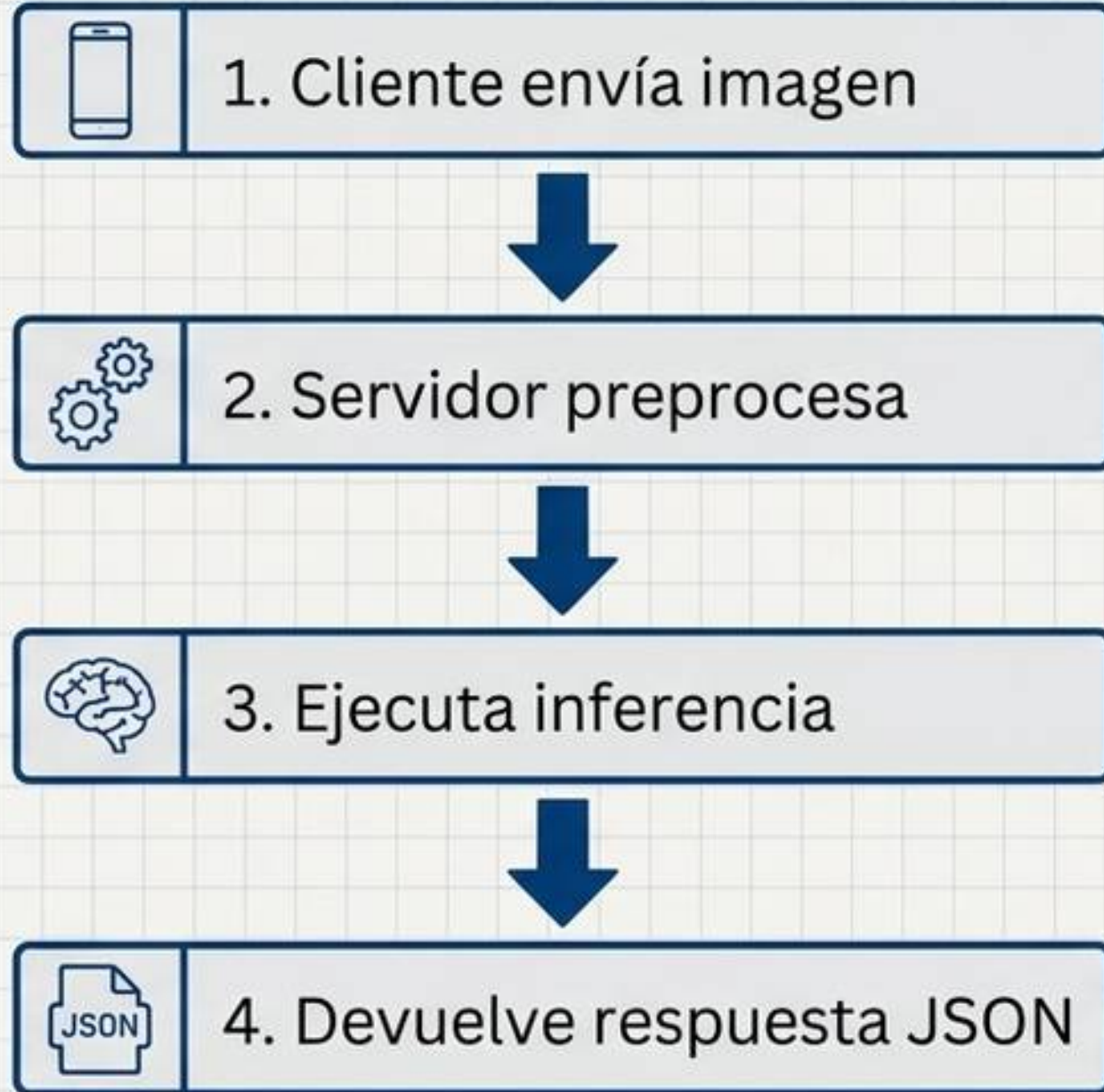
LA INTERFAZ API

Application Programming Interface. Funciona como un contrato estricto que oculta la complejidad del modelo y estandariza la comunicación.

EL ENDPOINT

El punto de contacto público operativo. Es la suma matemática de un Modelo Desplegado + una API Accesible.

ANATOMÍA DE UNA SOLICITUD (INFERENCIA VÍA FASTAPI)



```
@app.post("/predict")
def predict_image(file: UploadFile):
    # 1. Recibir imagen
    # 2. Preprocesar (Resize, Normalize)
    # 3. Ejecutar modelo (ONNX Runtime)
    return {
        "clase": "gato",
        "confianza": 0.95,
        "version_modelo": "v1.2"
    }
```

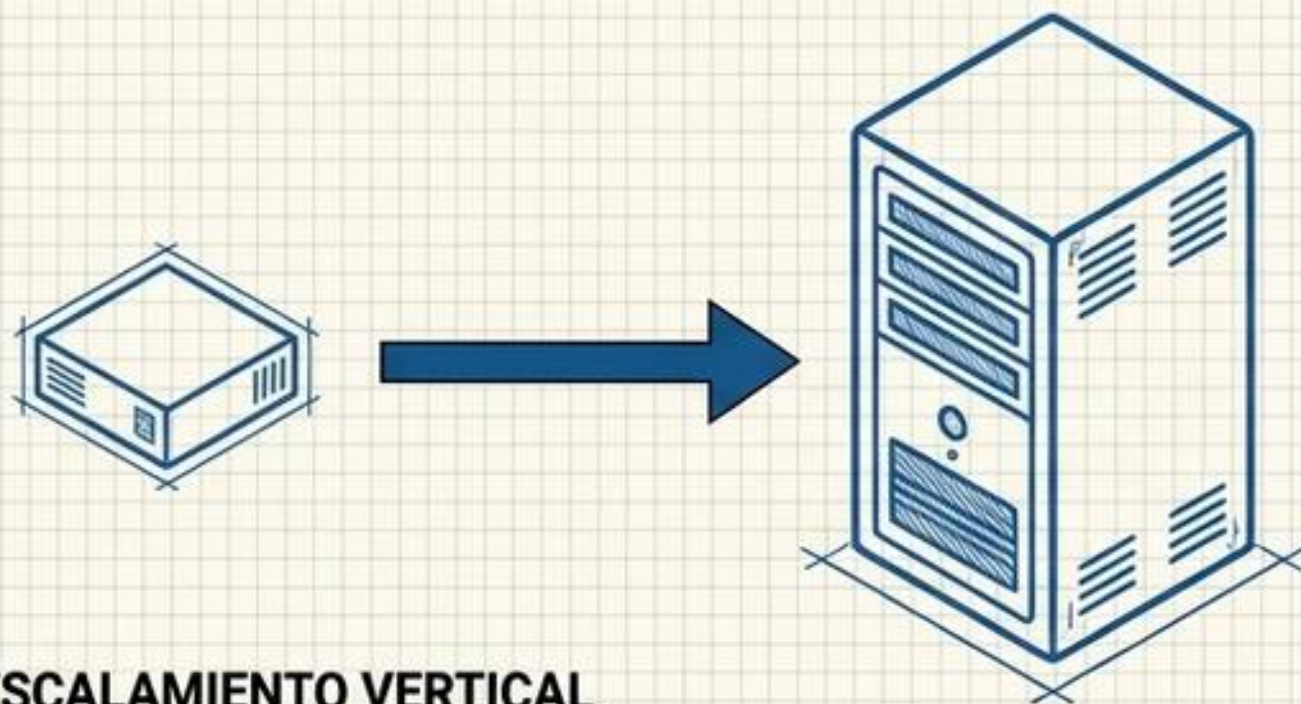
Ejemplo pedagógico de enrutamiento web utilizando FastAPI en Python.

DESAFÍOS DEL CLOUD: TIEMPO Y CAPACIDAD

$$\text{Latencia Total} = \text{Subida} + \text{Procesamiento} + \text{Inferencia} + \text{Respuesta}$$

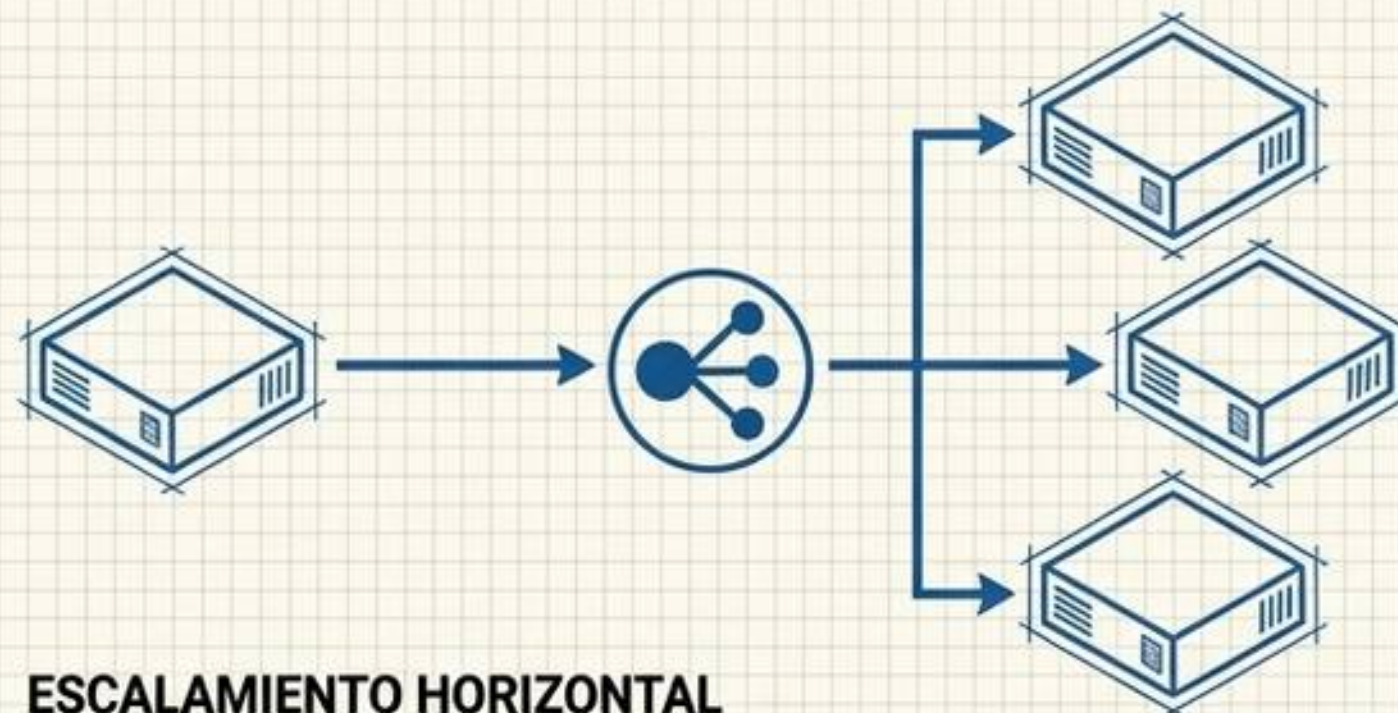


La ilusión de la latencia: La experiencia del usuario depende del tiempo total, no solo de la velocidad del modelo. La red suele ser el cuello de botella.



ESCALAMIENTO VERTICAL

Adaptar demanda mejorando la misma máquina (+RAM/GPU).

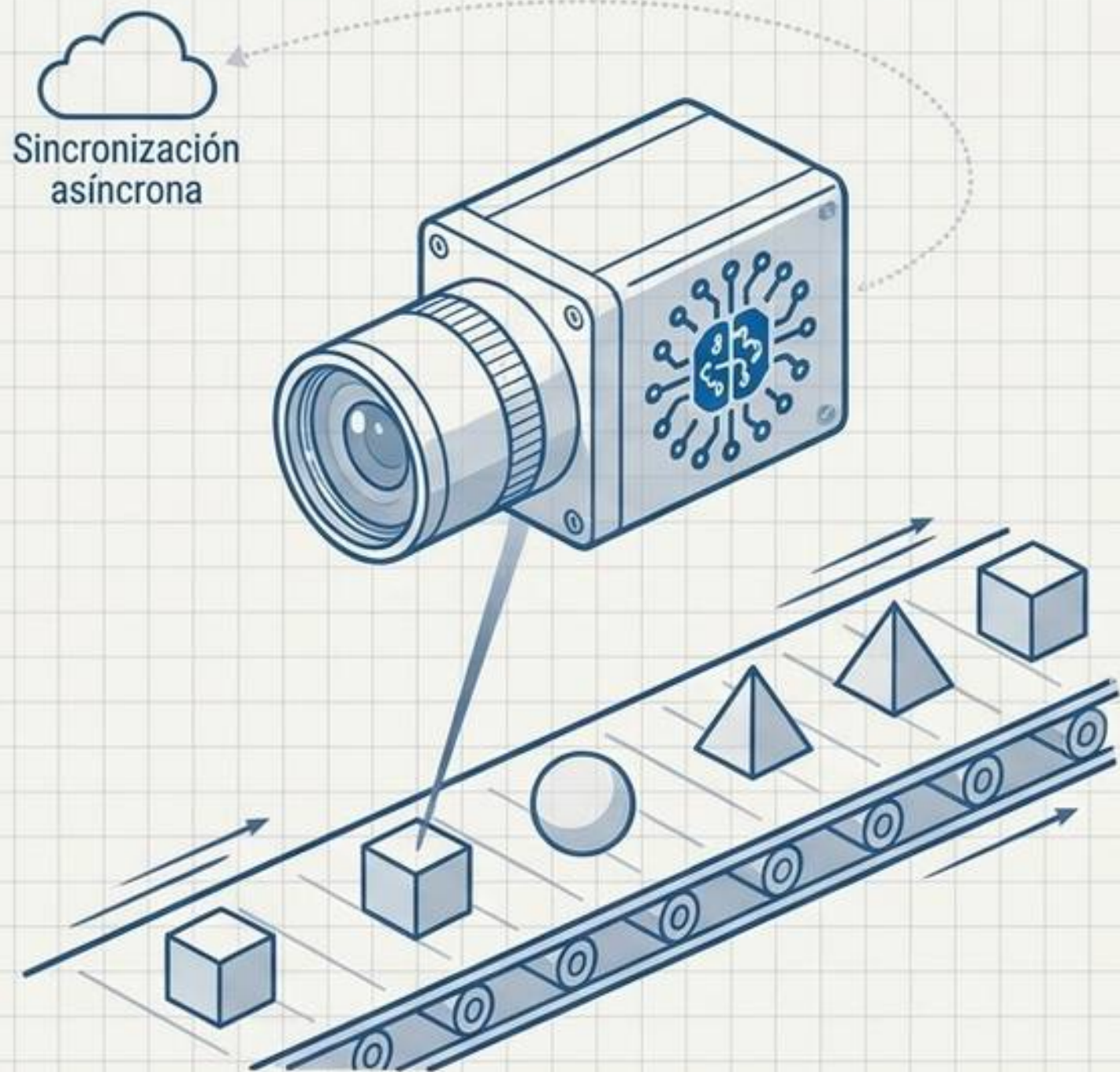


ESCALAMIENTO HORIZONTAL

Adaptar demanda multiplicando instancias en paralelo.

El Costo: Cada inferencia consume recursos. Un modelo optimizado (Semana 14) requiere menos hardware, reduciendo drásticamente los costos operativos.

EDGE COMPUTING: INTELIGENCIA EN EL BORDE



EL CONCEPTO

Procesar la información físicamente donde se genera (sensores, vehículos, móviles), sin delegar la inferencia a servidores centrales.

VENTAJAS CRÍTICAS

Respuesta inmediata (ultra baja latencia) vital para sistemas en tiempo real. Privacidad garantizada y ahorro masivo de ancho de banda.

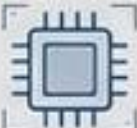
DESAFÍOS EXTREMOS

Los dispositivos Edge operan con recursos de CPU, memoria y batería severamente limitados. (Requiere técnicas de Cuantificación y Poda de la Semana 14).

ARQUITECTURA HÍBRIDA

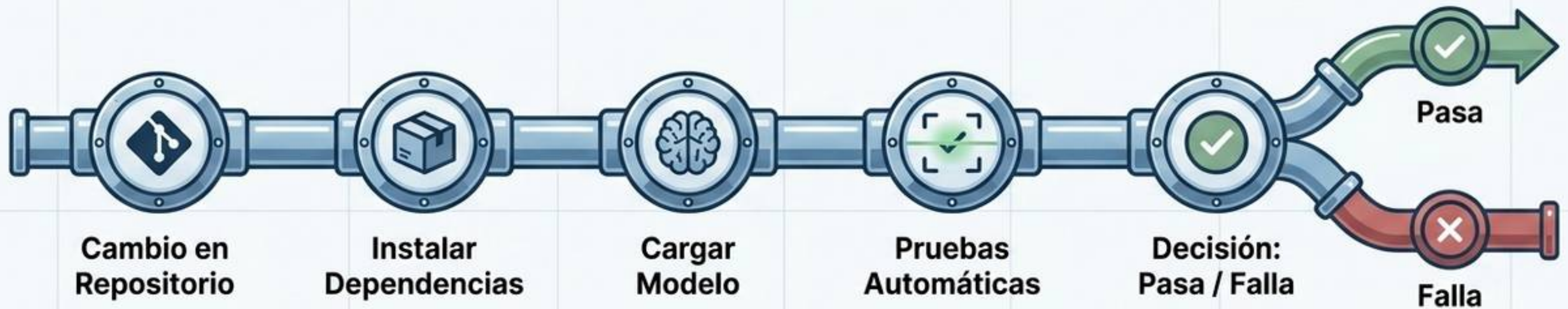
El Edge ejecuta la inferencia local de manera instantánea, mientras que la Nube (Cloud) recibe datos agregados de manera diferida para analítica y reentrenamiento.

COMPARATIVA DE ARQUITECTURAS: LOCAL VS. CLOUD VS. EDGE

CRITERIO DE EVALUACIÓN	LOCAL	CLOUD (NUBE)	EDGE COMPUTING
Conectividad a Internet 	No indispensable	Indispensable	Opcional (Capacidad offline)
Privacidad de Datos 	Absoluta (Alta)	Depende del servicio	Absoluta (Alta)
Recursos de Cómputo 	Depende del equipo	Ilimitados / Escalables	Severamente Limitados
Latencia de Red 	Nula	Significativa (cuello de botella)	Nula o mínima
Actualización de Modelo 	Manual / Compleja	Centralizada / Inmediata	Compleja (Flotas de dispositivos)

Conclusión: No existe una arquitectura universalmente superior; la elección debe responder de manera fundamentada al contexto del problema.

INTEGRACIÓN CONTINUA (CI): PROTEGIENDO LA PRODUCCIÓN



¿Qué es CI?

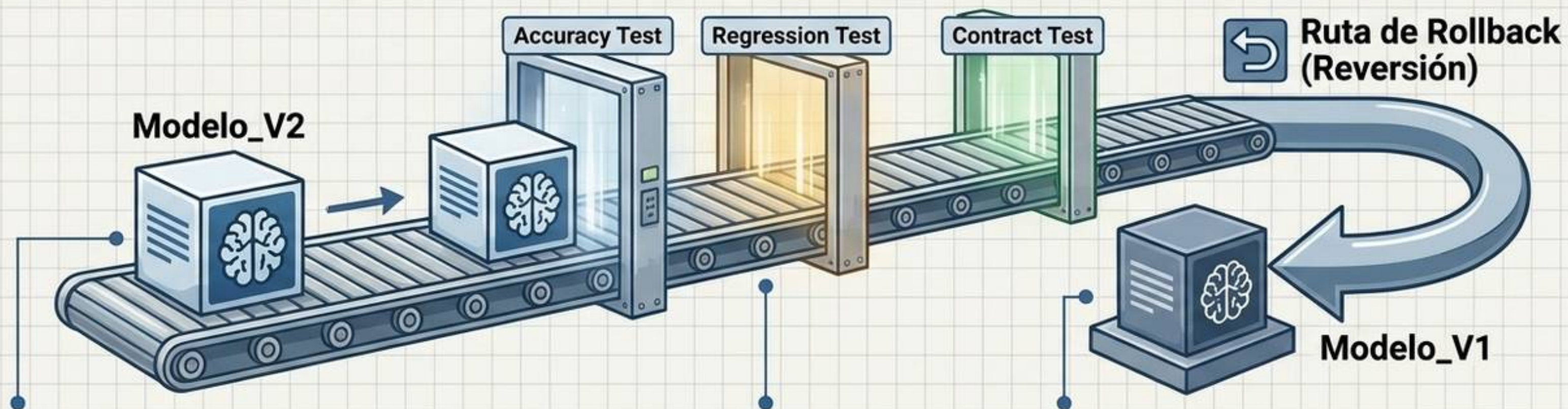
Una práctica defensiva. Los cambios se integran frecuentemente y se someten a validaciones automáticas para detectar errores tempranos, antes de afectar a los usuarios.

El Pipeline de ML vs Software

El software tradicional solo verifica que el código compile y funcione. En Machine Learning, el pipeline de CI verifica cuatro pilares simultáneamente: Código + Modelo + Dataset de prueba + Contrato de Entrada/Salida.

Automatización de validaciones en Machine Learning

El Principio Fundamental: Un nuevo modelo entrenado con más datos no reemplaza a la versión anterior automáticamente; primero debe probar su valor.



Regression Testing

Evalúa si el nuevo modelo empeora drásticamente el rendimiento en clases críticas donde la versión 1 funcionaba perfectamente.

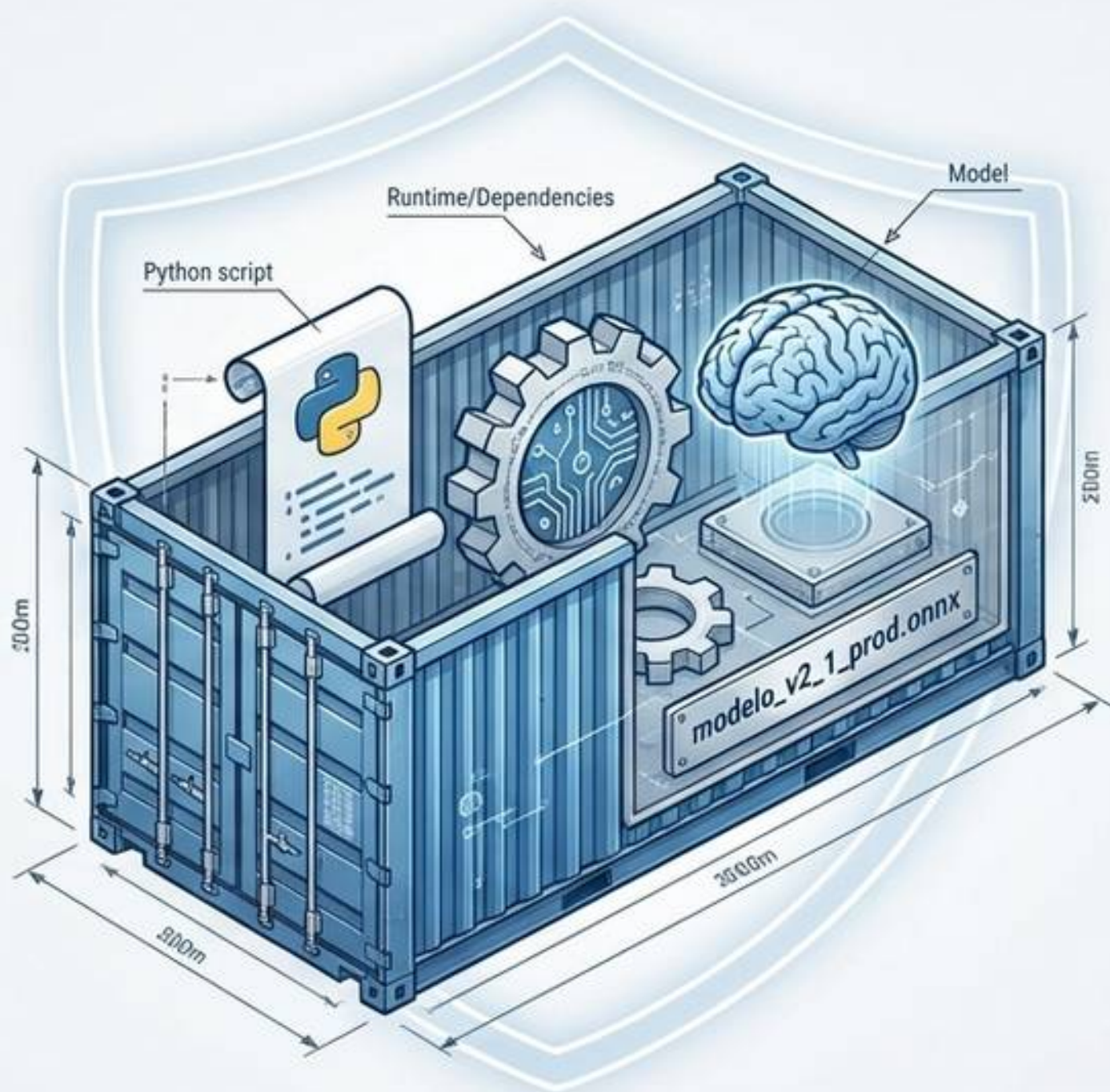
Prueba de Contrato

Asegura que el formato de los datos de entrada o salida (ej. estructura JSON) no haya mutado, lo cual rompería la aplicación del cliente.

Capacidad de Rollback

Si cualquier escáner falla, o si ocurren errores en producción, el sistema debe poseer la capacidad operativa de revertir de inmediato a la versión anterior estable.

Empaquetado, Contenedores y Versionado



Reproducibilidad Total

Un contenedor encapsula la aplicación, el runtime (ONNX, Python) y la configuración. Garantiza que el código funcione de manera idéntica en la laptop del desarrollador y en el servidor real.

Gestión de Dependencias

Es obligatorio fijar versiones exactas de las bibliotecas en un archivo requirements.txt para prevenir que futuras actualizaciones externas rompan el sistema.

El Modelo como Artefacto

Los modelos deben versionarse de manera explícita y trazable (ej. modelo_v2.onnx, nunca modelo_final.onnx). Esto permite responder auditablemente: "¿Qué modelo exacto generó esta predicción errónea ayer?"

Realidades de Producción: Seguridad y Data Drift

Degradación por Data Drift



Seguridad de la API



SEGURIDAD DE LA API



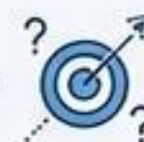
Proteger el endpoint definiendo límites estrictos (ej. tamaño máximo de imagen < 5MB) para evitar cargas maliciosas.

DATA DRIFT (DERIVA DE DATOS)



Ocurre cuando la distribución de los datos reales enviados por los usuarios difiere de los datos de entrenamiento (ej. nuevas cámaras, peor iluminación). El código está intacto, pero el desempeño decae.

CONCEPT DRIFT



Ocurre cuando cambia la definición misma o la relación de lo que se intenta predecir en el mundo real.

MONITOREO CONSTANTE (MLOps)

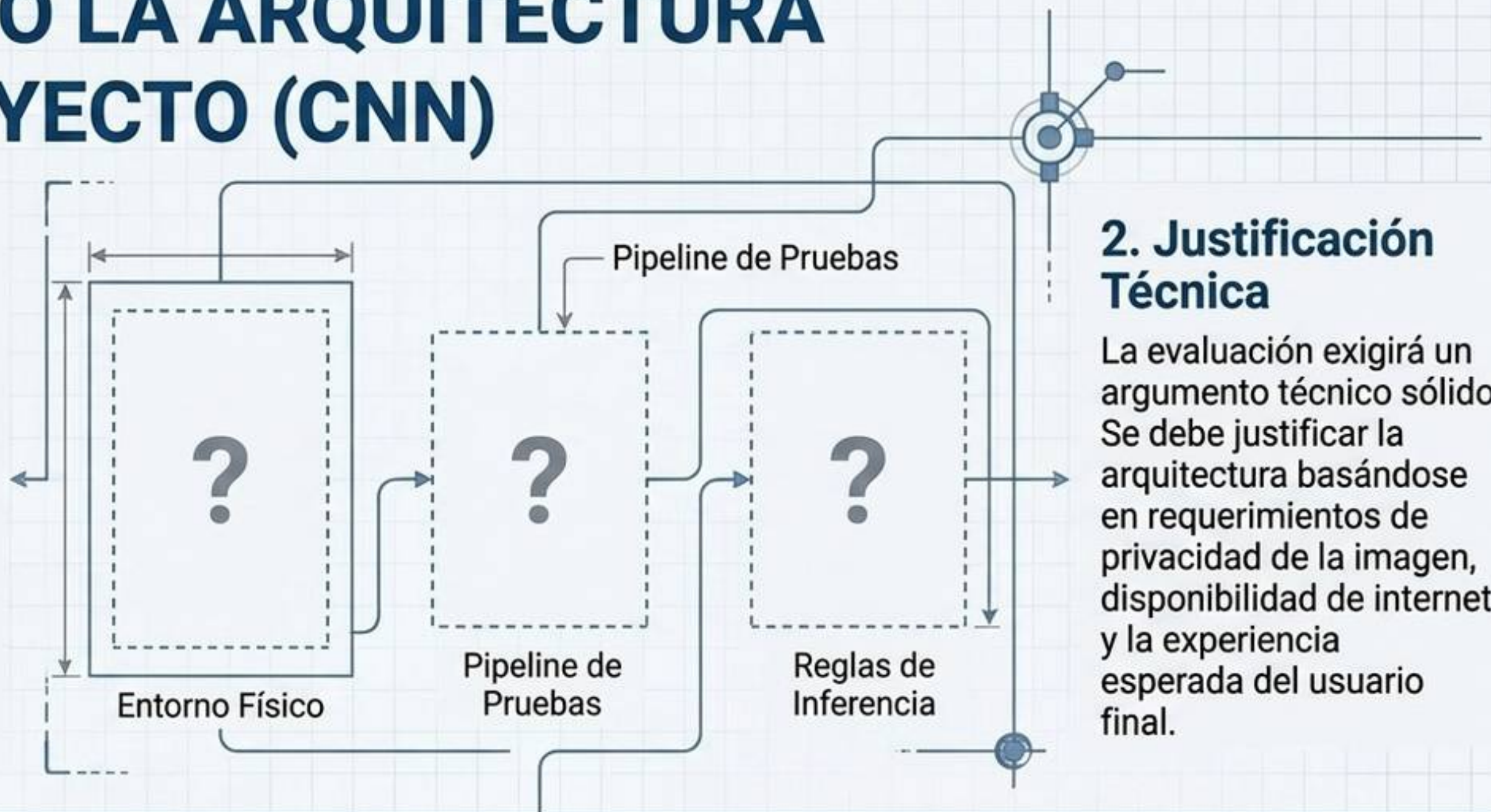


Desplegar un modelo no es el final. Se requiere vigilar continuamente las métricas de error y la distribución de clases para activar reentrenamientos.

DEFINIENDO LA ARQUITECTURA DE TU PROYECTO (CNN)

1. Decisión Arquitectónica

Cada dupla deberá elegir entre una solución de ejecución Local (ej. interfaz offline con ONNX) o una solución Cloud (ej. Interfaz Web + FastAPI backend). Ambas vías son válidas académicamente.



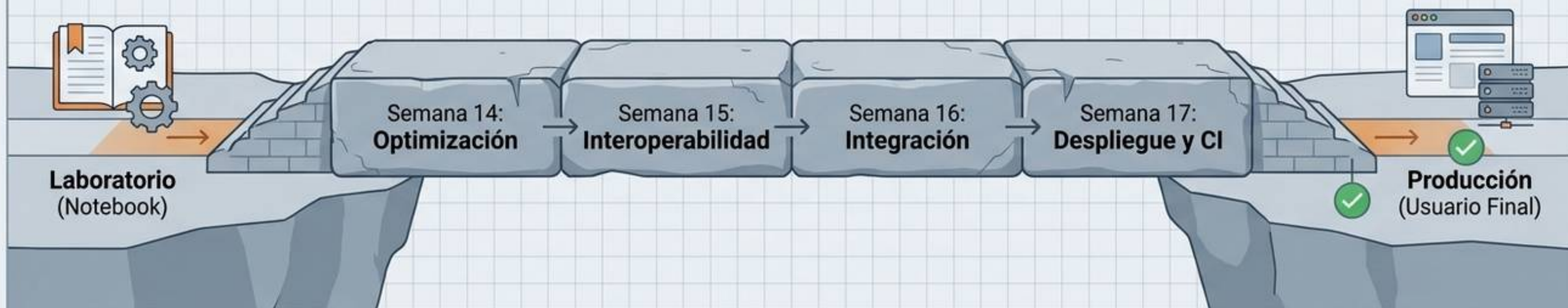
2. Justificación Técnica

La evaluación exigirá un argumento técnico sólido. Se debe justificar la arquitectura basándose en requerimientos de privacidad de la imagen, disponibilidad de internet y la experiencia esperada del usuario final.

3. Integración Mínima (CI)

Se espera la implementación de **pruebas automatizadas básicas y obligatorias** (por ejemplo, validación del contrato JSON de entrada/salida o filtros de rechazo de archivos inválidos) antes de considerar la solución desplegable.

SÍNTESIS Y EL ROMPECABEZAS COMPLETO



Conclusiones Clave:

- Desplegar es trasladar funcionalmente la solución matemática al usuario final.
- Local, Cloud y Edge ofrecen balances asimétricos entre conectividad, poder de cómputo y privacidad.
- Las APIs actúan como **barreras** que desacoplan la interfaz visual del cerebro (el modelo de inferencia).
- Las prácticas rigurosas de **CI y MLOps** previenen regresiones del software y preparan al sistema para combatir el inevitable Data Drift.



Hacia la Semana 18

Estos cuatro pilares transforman una red neuronal teórica (CNN) en un producto de software robusto, escalable y seguro. Estamos listos para finalizar el **Proyecto Integrador**.